IT UNIVERSITY OF COPENHAGEN

Game Programming 25

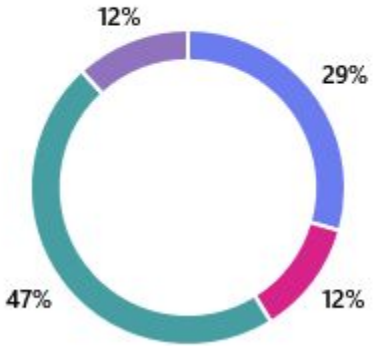
L10: Performance

Chris Carvelli

Topics decided!

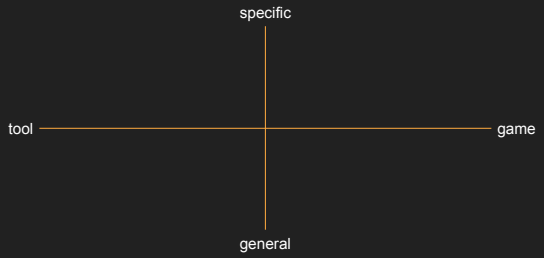
1. What topics would you like to see for the last two lectures of the course? (pick two)

Advanced Memory Management	5
Randomness and PCG	2
Data structures for Games	8
One extra supervision lecture	2

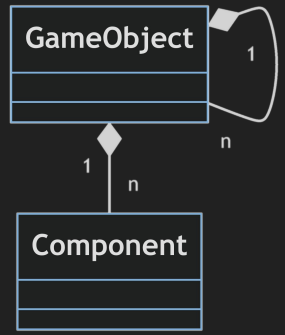


ITU Game Consoles Workshop

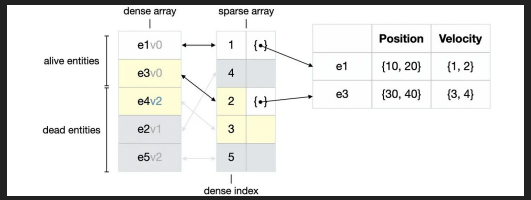
Recap



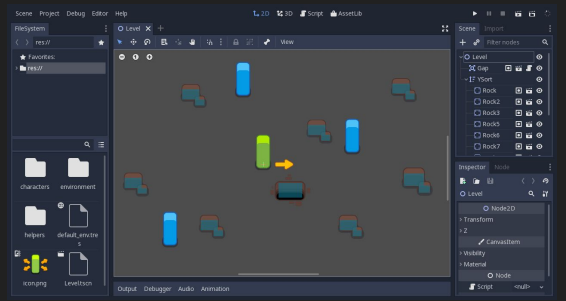
engine/game/tool



GameObject/Component



ECS



game<>engine layer

L10 - Performance

Agenda

1. Measurements and Data
2. Optimization types
3. Practical example

Agenda

1. Measurements and Data
2. Optimization types
3. Practical example

Importance of measuring

Task: sort an array.

Q: Who would win?

- selection sort
- bubble sort
- quick sort

Importance of measuring

Task: sort an array.

Q: Who would win?

- selection sort
- bubble sort
- quick sort

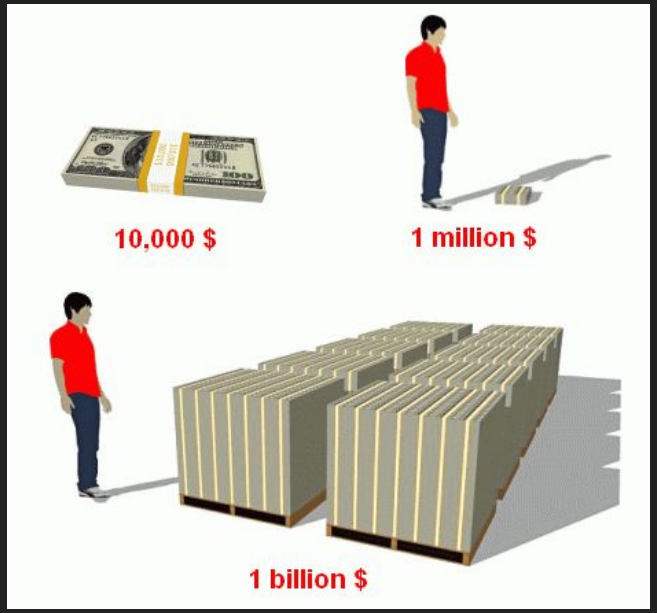
A: it depends!

- mostly, on how much data we have
- also, on the specific implementations
- finally, are there any regulariting in this specific sorting task?

In most cases we don’t know the answer to those questions (although we should always try to make an educated guess), so the only way to figure it out is measuring.

Time scales

Name	Graphics Model	# of CPU Cores	# of Threads	Max. Boost Clock	Base Clock
AMD Ryzen™ 9 5950X	Discrete Graphics Card Required	16	32	Up to 4.9 GHz	3.4 GHz
AMD Ryzen™ 9 5900XT	Discrete Graphics Card Required	16	32	Up to 4.8 GHz	3.3 GHz
AMD Ryzen™ 9 5900X	Discrete Graphics Card Required	12	24	Up to 4.8 GHz	3.7 GHz
AMD Ryzen™ 7 5800X3D	Discrete Graphics Card Required	8	16	Up to 4.5 GHz	3.4 GHz



3.4GHz
3.4 BILLION cycles per second*
One cycle takes 0.29 microseconds
*(modern CPUs run MANY instructions per cycle!)

Types of measuring

- sampling
- high frequency clock
- cycle counter

Sampling

CPU Usage

Current View: Call Tree | Expand Hot Path | Show Hot Path | Reset Root

Function Name	Total CPU [unit, %]	Self CPU [unit, %]	Module	Category
test_sorts.cpp (PID: 6136)	306 (100.00%)	0 (0.00%)	Multiple modules	
[External Call] ntdll.dll!0x00007ffa18b2c53c	305 (99.67%)	0 (0.00%)	ntdll	Kernel
mainCRTStartup	305 (99.67%)	0 (0.00%)	test_sorts.cpp	
__scrt_common_main	305 (99.67%)	0 (0.00%)	test_sorts.cpp	
__scrt_common_main_seh	305 (99.67%)	0 (0.00%)	test_sorts.cpp	
invoke_main	305 (99.67%)	0 (0.00%)	test_sorts.cpp	
main	305 (99.67%)	0 (0.00%)	test_sorts.cpp	
[External Call] ucrtbased.dll!0x00007ff9797a0dee	275 (89.87%)	275 (89.87%)	ucrtbased	
sort_selection_no_alloc	11 (3.59%)	5 (1.63%)	test_sorts.cpp	
sort_selection	11 (3.59%)	4 (1.31%)	test_sorts.cpp	
sort_selection_ints	6 (1.96%)	5 (1.63%)	test_sorts.cpp	
[External Call] ucrtbased.dll!0x00007ff9797c94ab	2 (0.65%)	2 (0.65%)	ucrtbased	
[External Call] ntdll.dll!0x00007ffa18b8910e	1 (0.33%)	1 (0.33%)	ntdll	Kernel

https://www.leeholmes.com/a-poor-mans-profiler-with-powershell-and-cdb/

https://poormansprofiler.org/

High Frequency Clocks

Windows

Acquiring high-resolution time stamps

- QueryPerformanceCounter
- QueryPerformanceFrequency

Unix

get_time() API family

- int clock_gettime (clockid_t clock, struct timespec *ts)
- int clock_getres (clockid_t clock, struct timespec *res)

std::chrono
std::chrono::system_clock
std::chrono::steady_clock

Cycle counters

Windows

__rdtsc()

Unix

get_cycle() API family

perf (tool)

WARNING!

these are very low level and can be misleading, always check the docs!

Pros and cons

- sampling
  - \+ very portable
  - \+ no instrumentation needed (but can be added for better context)
  - \+ unbiased
  - \+ runs in a setting closer to the final product
  - \- can be hard to interpret
  - \- shows mostly symptoms, not causes
  - \- data cannot be used by application
- high frequency clock
  - \+ measure very specific part of the code
  - \+ precise enough for most tasks
  - \+ can be used by the application (ie, time your main loop)
  - ± can be made portable
  - \- require prior setup to profile tasks
  - \- require instrumentation
- cycle counter
  - \+ highest resolution possible (not necessarily precision!)
  - \- all cons of high frequency clocks
  - \- least portable of them all
  - \- very difficult to interpret

Agenda

1. Measurements and Data
2. Optimization types
3. Practical example

Which performance problems
did you encounter?

Types of performance optimization

- asymptotical
- avoid useless work
- cache coherence
- cache results
- parallelization
- specialization and fast paths
- domain knowledge approximations

Cache coherence

- trendy, first one thrown around when talking about optimization
- refers to physical cache (ie, try to write code that can leverage hardware better)
- completely against human intuition (we have no idea how computers work)

```
int get_matching_entities(ITU_System* system, ITU_EntityId* out_entities)
{
    // get component/tag with least amount of entities
    // filter entities

    return system_ids_count;
}


void itu_sys_estorage_systems_update(SDLContext* context)
{
    for(int i = 0; i < ctx_estorage.systems_count; ++i)
    {
        ITU_System* system = &ctx_estorage.systems[i];
        ITU_EntityId ids[ENTITIES_COUNT_MAX];
        int system_ids_count = get_matching_entities(system, ids);

        // pass list of entity ids to the system,
        // it will take care of reading/writing data through pointers
        system->fn_update(context, system_ids, system_ids_count);
    }
}
```

Cache coherence

- trendy, first one thrown around when talking about optimization
- refers to physical cache (ie, try to write code that can leverage hardware better)
- completely against human intuition (we have no idea how computers work)

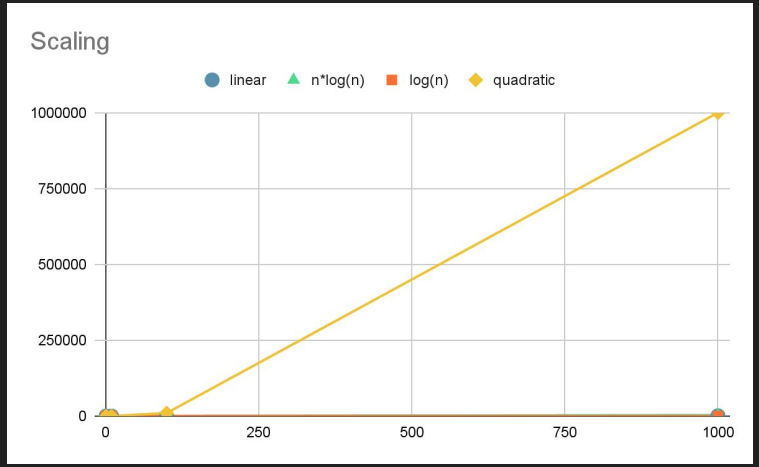
Faster!*

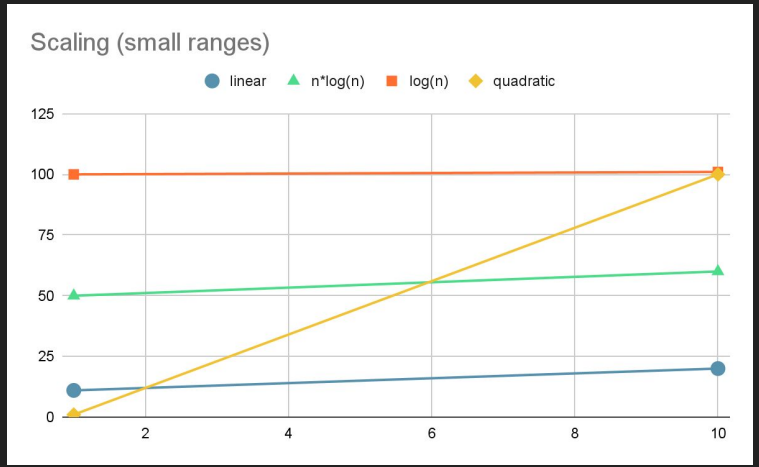
*(sometimes, depends on the amount and type of work, size and type of data, and so on…)

```
int get_data(ITU_System* system, ITU_EntityId* out_entities, void* out_data)
{
    // get component/tag with least amount of entities

    // while filtering entities, memcpy all relevant data in buffer
    // [
    //   [transform0, sprite0],
    //   [transform3, sprite42],
    //   [transform4, sprite42],
    //   [...],
    // ]

    return count;
}

void set_data(ITU_EntityId* entities, void* data, int count)
{
    // set new/updated data for each entity
}

void itu_sys_estorage_systems_update(SDLContext* context)
{
    for(int i = 0; i < ctx_estorage.systems_count; ++i)
    {
        ITU_System* system = &ctx_estorage.systems[i];
        ITU_EntityId entities[ENTITIES_COUNT_MAX];

        // `data_in` and `data_out` are probably a pre-allocated buffers
        int count = get_data(system, entities, data_in);

        system->fn_update(context, data_in, data_out, count);

        set_data(entities, data_in, count);
    }
}
```

Asymptotical

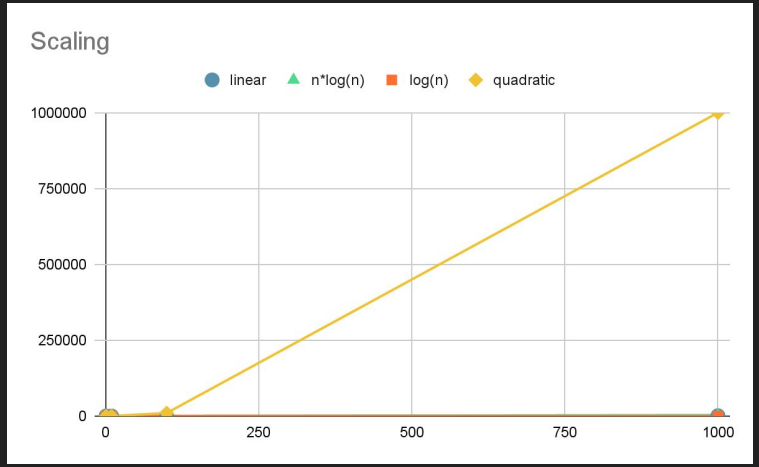
- another common approach
- asymptotic notation describes how algorithms scale "towards infinity"
- problem: we don’t have infinite elements
- most of the time, we don’t even have that many!
- it’s common for algorithms that are asymptotically better to be more complex

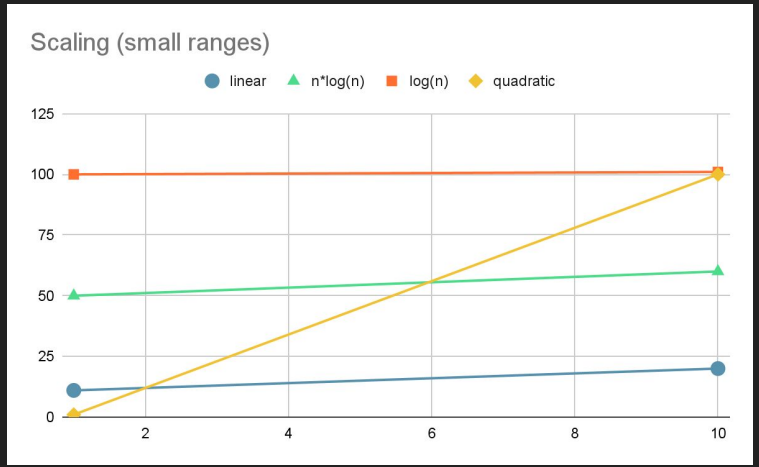




~~Asymptotical~~ Appropriate algo!

- another common approach
- asymptotic notation describes how algorithms scale "towards infinity"
- problem: we don't have infinite elements
- most of the time, we don't even have that many!
- it's common for algorithms that are asymptotically better to be more complex



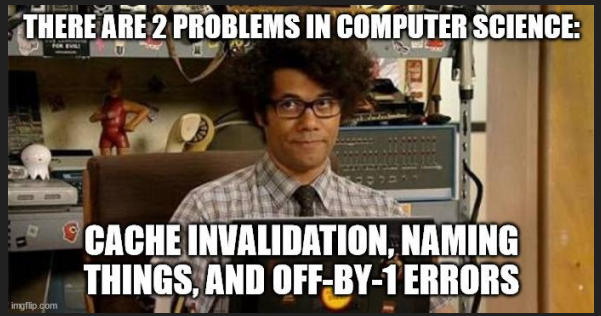


Avoid useless work

- different from "git gud bro"
- "superfluous" probably a better term
- usual causes:
  - code reuse
  - over-aggressive error checks
  - too many levels of abstractions

Cache results

- refers to software cache (ie, try to write code that avoids redoing complex computation
- as before, patterns may not be easy to spot, especially in big codebases with a lot of abstraction levels
- introduces the problem of reading data from a cache that are not valid anymore (cache invalidation)




```
struct ITU_System
{
    const char* name;
    ITU_Component* components[SYSTEM_COMPONENTS_MAX];
    int components_count;

    ITU_TagType tags[SYSTEM_TAGS_MAX];
    int tags_count;

    ITU_SystemUpdateFunction fn_update;
};

void itu_sys_estorage_add_system(ITU_SystemDef system_def)
{
    ITU_System* sys= &ctx.systems[ctx.systems_count++];

    // build component pool pointers
    // (this requires component pools to be already set up)
    for(int j = 0; j < COMPONENTS_COUNT_MAX; ++j)
    {
        Uint64 component_bitmask = 1ll << j;
        if(system_def.component_mask & component_bitmask)
            sys->components[sys->components_count++] = ctx.components[j];
    }
    for(int j = 0; j < TAGS_COUNT_MAX; ++j)
    {
        Uint64 tag_bitmask = 1ll << j;
        if(system_def.tag_mask & tag_bitmask)
            sys->tags[sys->tags_count++] = j;
    }
    sys->fn_update = system_def.fn_update;
    sys->name = system_def.name;
}
```

Parallelization

- can mean many things, but in games we care mostly about
  - multi-threading (parallelized code)
  - SIMD (parallelized instructions)

- other examples
  - coroutine/fibers (fancy new lightweight threads)
  - multi-processing (sort of superseded by multi-threading)
  - multiple machines (cloud computing, mostly used for Continuous Integration and offline render farms)

Parallelization

- can mean many things, but in games we care mostly about
  - ~~multi-threading (parallelized code)~~
  - SIMD (parallelized instructions)

central focus of High Performance Game Programming (GAMES, 3rd semester)

- other examples
  - coroutine/fibers (fancy new lightweight threads)
  - multi-processing (sort of superseded by multi-threading)
  - multiple machines (cloud computing, mostly used for Continuous Integration and offline render farms)

SIMD

- Same Instruction Multiple Data
- intrinsics that expose specialized registers in the CPU that can perform (mostly) mathematical operations in parallel
- different CPUs implement different subsets
- need to write code so that it CAN leverage them, but DOESN'T HAVE TO

AVX512VNNI	19.91%	+0.34%
SHA	75.45%	-0.31%
FMA	94.81%	-0.29%
SSE2	99.94%	0.00%
NTFS	94.75%	-0.56%
BMI2	94.49%	-0.29%
AVX	96.99%	-0.25%
LAHF / SAHF	99.93%	-0.01%
SSE4.2	99.76%	-0.02%
SSE4a	41.96%	+0.73%
SSE4.1	99.81%	-0.01%
SSSE3	99.84%	-0.02%
SSE3	99.94%	0.00%

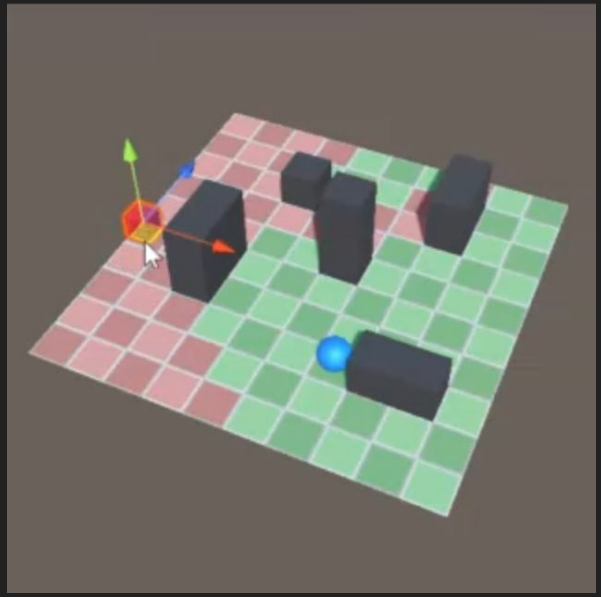
Specialization and fast paths

- while it is rarely possible to have solutions that are simple, generic and performant, we can get very close where it matters
- simple and generic are a great way to kickstart a project/prototype, so that then we can start gathering data
- usually, the first approach will be good enough in most cases, so that specialized solutions can be applied only where needed

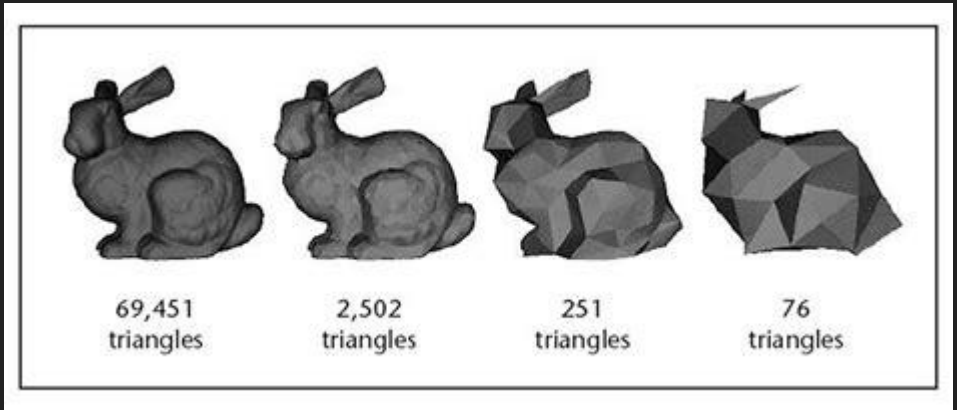
- examples:
  - stdlib algorithms VS custom implementations
  - transform components VS dedicated transform tree structura
  - acceleration structures in general

Domain knowledge approximations

- sometimes, "correct" is not necessary
- examples:
  - UI processes input from LAST frame
  - AI pathfinding does not run on most up-to-date world state
  - objects that are far away from the camera can be rendered at lower resolutions



https://allenchou.net/2021/05/delayed-result-gathering/



Agenda

1. Measurements and Data
2. Optimization types
3. Practical example